

ProxCode: Efficient Proximity Searchable Encryption from Error Correcting Codes

Maryam Rezapour*

Benjamin Fuller†

April 10, 2026

Abstract

This work builds approximate proximity searchable encryption. Secure biometric databases are the primary application. Prior work (Kuzu, Islam, and Kantarcioglu, ICDE 2012) combines locality-sensitive hashes, or LSHs, (Indyk, STOC '98), and secure multimaps. The multimap associates LSH outputs as keywords to biometrics as values.

When the desired result set is of size at most one, we show a new preprocessing technique and system called **ProxCode** that inserts shares of a linear secret sharing into the map instead of the full biometric. Instead of choosing shares independently, shares are correlated so exactly one share is associated with each keyword/LSH output. As a result, one can rely on a map instead of a multimap. Secure maps are easier to construct with lower leakage than multimaps.

This approach reduces the required number of LSHs for a fixed accuracy for many parameters. Our scheme improves most when combining a high accuracy requirement with a biometric with large underlying noise. Our approach builds on any secure map. To benchmark, we implement the scheme using the tree based oblivious map of Wang et al. (CCS, 2014) and evaluate efficiency and accuracy for iris, synthetic, and random data.

Compared to the recent work of Ha et al. (Codaspy 2025), for the largest parameters tested and a comparable true accept rate, our work reduces the number of LSHs by a factor of 4.3. This reduction impacts all efficiency metrics; our rounds are smaller by 2, storage by a factor of 6.5, and parallel time by a factor of 8.

1 Introduction

This work builds approximate proximity searchable encryption, called APSS [KIK12,BT21,HCD⁺25,FWG⁺16,WYLH14,LPW⁺20]. Secure biometric databases [BBOH96,Dau14,Fou] are a major application of APSS. Searchable encryption considers two parties a client, denoted as **Client**, and a server, denoted as **Server**. The goal of the searchable encryption is for **Client** to outsource a database, $\mathcal{DB}$, to **Server** and be able to query this outsourced database. The system should limit what is learned by a semi-honest **Server** while still allowing the client to retrieve the relevant results from the database. See prior reviews [BHJP14,FVY⁺17,KKM⁺22,RW23,IKK12] and work on searchable encryption [CGPR15,KKNO16,WLD⁺17,GSB⁺17,GLMP18,KPT19,MT19,KE19,KPT20,FMC⁺20,FP22,GPP23,APP⁺23,HKR⁺24].

We now set notation of APSS and provide a review of prior work. Let $\mathcal{DB} = x_1, \dots, x_M$ be a collection of records. For metric $\mathcal{D}$, a distance parameter t , and query y , the goal of the search is to find the set $\text{Res} = \{x_i \in \mathcal{DB} | \mathcal{D}(x_i, y) \leq t\}$. Our system is designed for 1-to- n matching, for query y , one is trying to retrieve the single record $x_i \in \mathcal{DB}$ such that $\mathcal{D}(y, x_i) \leq t$. Most work in APSS uses preprocessing techniques and encrypted multimaps.

Encrypted Multimaps Multimaps [KM19], MM, allow association of keywords with values x_i .¹ An encrypted multimap is the same except there are interactive protocols between the **Client** and **Server**. We

*University of Connecticut. Email: maryam.rezapour@uconn.edu.

†University of Connecticut. Email: benjamin.fuller@uconn.edu.

¹A multimap has two operations: 1) $\text{MM.Init}(\{\text{keyword}, \text{value}\})$ that takes a set of **keyword**, **value** pairs and associates **value** with **keyword**, and 2) $\text{Resp} \leftarrow \text{MM}[\text{keyword}]$ which returns all values associated with **value**.

use the following notation for the encrypted version (with the Client’s inputs and outputs on the top row and the Server’s inputs and outputs on the bottom row):

$$\begin{pmatrix} K \\ \text{EMM} \end{pmatrix} \leftarrow \text{MM.Setup} \left(\begin{pmatrix} \text{MM} \\ \perp \end{pmatrix} \right)$$

$$\begin{pmatrix} \text{MM}[\text{keyword}] \\ \perp \end{pmatrix} \leftarrow \text{MM.Search} \left(\begin{pmatrix} \text{keyword}, K \\ \text{EMM} \end{pmatrix} \right)$$

The goal of preprocessing is to create accurate search while minimizing the size of the multimap, which impacts storage, bandwidth, computation, and communication rounds.

Preprocessing Data for APSS Prior work [KIK12, BT21, HCD⁺25] use locality sensitive hashes or LSHs [IM98] to preprocess. We provide a brief overview of LSHs and then explain how they are used for preprocessing. LSHs map close items to the same value more frequently than they map far items to the same value. For some n number of LSHs, a multimap MM , and LSH family LSH , the Client proceeds as follows, $\text{APSS.Init}(\mathcal{DB} = x_1, \dots, x_M)$:

1. Sample n LSHs, $\text{LSH}_1, \dots, \text{LSH}_n \leftarrow \text{LSH}$.
2. For $j = 1, \dots, n$ & $i = 1, \dots, M$:

$$\text{MM.add}(\text{keyword} = (j, \text{LSH}_j(x_i)), \text{value} = x_i).$$

3. Store MM on Server by executing

$$\begin{pmatrix} K \\ \text{EMM} \end{pmatrix} \leftarrow \text{MM.Init} \left(\begin{pmatrix} \text{MM} \\ \perp \end{pmatrix} \right).$$

Then, $\text{APSS.Search}(y)$ is:

1. Client computes $\text{LSH}_1(y), \dots, \text{LSH}_n(y)$.
2. For $i = 1$ to n :

$$\begin{pmatrix} \text{Resp}_i \\ \perp \end{pmatrix} \leftarrow \text{MM.Search} \left(\begin{pmatrix} \text{LSH}_i(y), K \\ \text{EMM} \end{pmatrix} \right).$$

3. Output all returned records across all queries.

There are two aspects to understand for the above construction for a fixed number of LSH’s n : security and accuracy. We discuss these issues below.

Security. The security of the above construction derives from the encrypted multimap. Constructions of (dynamic) multimaps [SWP00, KMO18, GKM21, GPP23, AG22, RW23, APP⁺23, WSL⁺22, PPYY19]² have nonzero leakage including query equality. Patel et al. [PPYY19] showed that avoiding query equality leakage for multimaps requires high overhead techniques similar to ORAM.

Accuracy. LSH-based preprocessing solutions have imperfect accuracy. The two accuracy parameters are:

1. δ_{Close} measures how frequently the close record is not returned, and
2. δ_{Far} measures what fraction of the database is (incorrectly) returned.

High accuracy systems have three advantages:

²This generation of low leakage maps followed attacks on the prior generation of map constructions [CGPR15, KKNO16, ZKP16, GLMP18, GLMP19, GJW19, KPT20, DHP21, OK21].

1. Returned records are more likely to be relevant. Many leakage attacks [CGPR15] are based on identifying the correlation between returned values, so decreasing erroneously returned values improves security.
2. A decrease in the maximum number of values associated with a keyword, a key efficiency metric for secure multimaps. If this is made to a small constant, one can use a map instead.
3. In a three party system where the querier does not know the whole dataset it reduces unintentional exposure of biometrics (discussion in Section 1.2).

1.1 Our Contribution

This work contributes 1) a data preprocessing method for accurate proximity search, 2) an implementation of this processing technique using an oblivious tree-based map [WNL⁺14], and 3) an evaluation of accuracy and efficiency on iris and random data. Our approach is to transform the query from a disjunction, where one concatenates the list of all records that match some LSH, to a k -out-of- n query, where one retrieves at most a single record that matches at least k LSHs. We call the system **ProxCODE** for efficient proximity search from error correcting codes.³ The high-level approach proceeds in five steps:

1. Compute n LSHs and for every record x , compute $\text{LSH}_i(x)$ for $1 \leq i \leq n$.
2. For each x_i , define $P_{x_i} = \{p \mid \text{degree}(p) \leq k \wedge p(0) = x_i\}$,⁴
3. For each pair $1 \leq i < j \leq M$ do the following:
 - (a) If p_{x_i} has not been defined, set $p_{x_i} \leftarrow P_{x_i}$ uniformly.
 - (b) For each $1 \leq \beta \leq n$ if $\text{LSH}_\beta(x_i) = \text{LSH}_\beta(x_j)$ remove those $p_j \in P_{x_j}$ such that $p_i(\beta) \neq p_j(\beta)$.
 - (c) If $P_{x_j} = \emptyset$, go to step 1.
4. Set p_{x_M} as a uniformly random element of P_{x_M} .
5. Associate in the map for all i, j

$$\text{Map}[(j, \text{LSH}_j(x_i))] = p_{x_i}(j).$$

At search time, the client queries the map for all $(j, \text{LSH}_j(x'))$, collects results from the map, and uses polynomial interpolation to reconstruct the relevant record.

Roughly, instead of associating x_i with LSH values, we associate polynomial points. If a querier gets enough points on the same polynomial, they can reconstruct the original value. Each LSH collision between records x_i, x_j of the database, where $i < j$, removes a degree of freedom from the polynomial of x_j , denoted as P_{x_j} ; but does not require one to associate multiple values with the same LSH keyword. The procedure succeeds as long as no record x_j has no more than $k - 1$ LSH collisions with other x_i (more precisely, more than $k - 1$ distinct LSHs have collisions).

Improving n for fixed accuracy Our analysis shows that for the error regimes present in biometrics, we can sample a set of M polynomials using fewer LSHs than the Baseline disjunctive search's requirement described by the recent work of Ha et al. [HCD⁺25].

Mean FHD (Fractional Hamming Distance) is the Hamming distance divided by the length of the vector. FHD for biometrics varies between 10% – 30% depending on the biometric, collection conditions, and the feature extractor. Ha et al. [HCD⁺25] point out it is necessary to consider distance higher than the mean of biometric error rate to achieve low δ_{close} . For one accuracy regime, their analysis suggests $n = 80$ LSHs suffices to capture the distribution mean but $n \approx 1000$ LSHs are needed to capture the distribution tail.

³In the majority of our discussion, we use the notation of Reed-Solomon codes, as we discuss in the Conclusion, it may be possible to achieve better parameters with custom codes. We use map notation instead of multimap notation. In our system one only associates one value with each keyword.

⁴The system works perfectly well if one associates the value i and uses a separate mechanism to retrieve x_i from i . Associating x_i prevents a second lookup with associated leakage [GPPW24, GPP23].

Error Rate	Improvement ($\log_{10}(n)$)				
	.10	.15	.20	.25	.30
$\delta_{\text{Far}} = 10^{-3}$	-0.5	-0.2	0.3	2.1	1.3
$\delta_{\text{Far}} = 10^{-4}$	-0.2	0.1	0.7	1.6	2.8
$\delta_{\text{Far}} = 10^{-6}$	0	0.6	1.4	2.4	3.9

Table 1: Summary of improvement in number of required LSHs across biometric error rates ($1 - \epsilon'_t$) and accuracy of the scheme with respect to false accepts denoted as δ_{Far} , fully described in Appendix A. We note the substantial improvement for the high error rate regime. Dataset of size $M = 10^4$.

Theoretic Improvement We show theoretical improvement in the number of required LSHs in Table 1 for a dataset of size 10,000. This directly translates to the overall size of the $|\text{Map}|$ that must be stored. Many cryptographic constructions have storage of $\omega(|\text{Map}|)$.

The accuracy level of δ_{Far} presented in Table 1 has a slightly different meaning for the baseline and **ProxCode**. Roughly for **ProxCode** it is the probability of a query returning a far value. For the baseline, it is the fraction of far records that are returned. Our efficiency improvements are highest for high-accuracy regimes with large underlying biometric noise.

Concrete Improvements Table 2 shows a concrete comparison between representative parameters for synthetic and random data for Ha et al. and this work. We focus our parameters on achieving TAR of $\approx .90$. On a random dataset of the same size, **ProxCode** reduces the number of required LSHs by a factor of 4.3. For synthetic data, we are able to reduce the number of LSHs by nearly a factor of 2 on a dataset 5 times larger.

Improving the number of LSHs, n , directly results in a smaller map. A smaller map means less storage, fewer rounds for our tree-based OMap construction, and faster parallel search. As we discuss in Section 5, Ha et al. use a filtering technique to search fewer items in the map, this technique works in our setting as well but is not reported. As a result, parallel time is the appropriate comparison.

Security and Cryptographic overhead **ProxCode** can be secured using any map. Our implementation and efficiency results consider an oblivious tree-based map [WNL⁺14]. We consider security when instantiated with 1) a map that leaks search [LZWaT14, OK21] & access pattern [SWP00, CGKO06] and 2) an oblivious map [Mic97, WNL⁺14, BT21]. When the map reveals query and access pattern, our scheme reveals the query and access pattern for each individual search term which we call subquery and subaccess pattern leakage using the language of Falzon et al. [FMET22]. If one uses a map with access pattern leakage, subquery and subaccess leakage are in 1-1 correspondence. For our real iris data, we observe roughly 3000 repeats on 200 queries when $n = 100$. This is on queries of different irises. One expects much higher subquery equality if multiple readings of the same iris are in the set of queries.

We present a prototype implementation of the above **ProxCode** scheme integrated into an implementation of Wang et al.’s [WNL⁺14] oblivious map (Github). We evaluate accuracy and overhead on

1. The IITD iris data set [KP10] for realism using a feature extractor designed for an LSH-based fuzzy extractor [SDF⁺25],
2. A synthetic dataset designed to have the same degrees of freedom as the above feature extractor, and
3. Random data.

For the synthetic and random data, the number of errors follows the distribution from real data. Table 2 compares the largest tested parameters between **ProxCode** and Ha et al. [HCD⁺25]. The main comparison point is the number of LSHs. Improving this improves all other values. See further results in Section 5.

1.2 Implications for Client Security

Throughout the body of this work, we define a traditional two-party setting of searchable encryption where there is a data owner that outsources data to a server. Some searchable encryption systems operate in the three party setting where there is a data owner, server, and a client [FMC⁺15, HSWW18, WP21]. In this

System	Data Type	M	LSHs n	Map	Setup	Storage	Search		
							Seq.	Par.	TAR
[HCD ⁺ 25]	Synthetic	5000	1300	6.5M	12.5 hrs	35.6 GB	8.8	.023	.83
	Random	25000	3500	87.5M	153 hrs	285 GB	7.9	.035	.91
ProxCode	Synthetic	25000	800	20M	6.6 hrs	43.5 GB	7.2	.0042	.88
	Random	25000	800	20M	6.8 hrs	43.5 GB	7.5	.0051	.88

Table 2: Comparison of largest tested parameters. In Ha et al. [HCD⁺25] there is a filtering phase that finds non-null values and only searches for 72/20 (for synthetic and random respectively) items in the map. As we discuss in Section 5, if one implements a similar phase in our scheme we estimate the sequential time to be .26 seconds. The fairest comparison is parallel search time which is agnostic to such an optimization.

setting, the database contents and queries are both private. In this setting, the data owner gives the client a token that allows them to execute their query. Our scheme shows that the client is unlikely to gain enough code symbols to learn anything about any records far from their query. In the body, we measure for real data how many code symbols are gained by a client across multiple queries. We compare this to the number of biometrics that are completely leaked using the baseline setting (Section 5.3). In ProxCode, learning anything about a non-queried biometric requires a persistent adversary [GRS17] with many queries.

Further Prior Work. Boldyreva and Tang considered the related problem of zero-leakage k -nearest-neighbor search [BT21]. In (approximate) k -nearest neighbors, the goal is to retrieve the k closest records [WHL16, WHL20, BT21]. There have been leakage abuse attacks against k -nearest neighbor systems that reveal access pattern [KPT19, KPT20, LMWY20, CCD⁺20].

Organization. Section 2 introduces preliminary notation including the definition of APSS. Section 3 introduces the Baseline construction described in the Introduction and describes the required number of LSHs for fixed accuracy parameters. Section 4 presents ProxCode and proves correctness and security. Section 5 presents our implementation and evaluation. Section 6 concludes and discusses future work. Appendix A gives theoretical parameter regimes, this analysis assumes LSHs have exactly ϵ_f and ϵ_t matching on random data, ignoring the variance in these parameters.

2 Preliminaries

Throughout this work we use the following notation:

1. Let λ be a security parameter.
2. Let stored records x_i be values over $\{0, 1\}^\gamma$.
3. Let $M = |\mathcal{DB}|$, the number of records.
4. We consider prime fields over prime power p , denoted $\mathbb{F}_p$.
5. For a Reed Solomon code, let k be the dimension, $k_{correct}$ be the required number of correct symbols, and k_{error} be the maximum number of incorrect symbols. See Definition 2.
6. Let n denote the number of LSHs, and,
7. If one uses an extended LSH, let α denote the number of concatenated LSHs.

We use $z = (z_1, \dots, z_\ell)$ to denote a vector. We focus on the Hamming distance. For vectors $x, y \in \{0, 1\}^\gamma$, let $\mathcal{D}(x, y) = |\{i | x_i \neq y_i\}|$ denote the Hamming distance between x and y . The Hamming distance metric is frequently used in iris recognition [Dau09]. Our techniques apply to any metric with locality sensitive hashes [IM98]. For a positive integer x , let $[x]$ denote the set $\{1, \dots, x\}$. TAR stands for True Accept Rate; FAR stands for False Accept Rate. For a protocol, Prot, between a client Client and a server Server we use notation

$$\begin{pmatrix} o_{\text{Client}} \\ o_{\text{Server}} \end{pmatrix} \leftarrow \text{Prot} \begin{pmatrix} i_{\text{Client}} \\ i_{\text{Server}} \end{pmatrix}$$

with $i_{\text{Client}}, o_{\text{Client}}, i_{\text{Server}}, o_{\text{Server}}$ denoting the client's and the server's inputs and outputs respectively. Protocols are written from the client's perspective.

Definition 1 (Locality-sensitive Hashing (LSH)). *Let $t \in \mathbb{N}$, $c > 1$ and $\epsilon_t, \epsilon_f \in [0, 1]$ with $\epsilon_t > \epsilon_f$. $\mathcal{H}$ defines a $(t, ct, \epsilon_t, \epsilon_f)$ -sensitive hash family if for any $x, y \in \{0, 1\}^\gamma$ one has:*

- *If $\mathcal{D}(x, y) \leq t$ then $\Pr_{h \leftarrow \mathcal{H}}[h(x) = h(y)] \geq \epsilon_t$*
- *If $\mathcal{D}(x, y) \geq ct$ then $\Pr_{h \leftarrow \mathcal{H}}[h(x) = h(y)] \leq \epsilon_f$*

where $\mathcal{D}(x, y)$ denotes the Hamming distance between binary vectors x and y .

An extended LSH is formed by concatenating α independently sampled LSHs. This output is an LSH, with parameters $\epsilon_t = \epsilon_t^\alpha$ and $\epsilon_f = \epsilon_f^\alpha$. This is used to compute parameters but can just be considered an LSH with smaller ϵ_t, ϵ_f .

Definition 2 (Reed-Solomon Codes). *An (n, k) Reed-Solomon code over $\mathbb{F}_p$ is*

$$\text{RS}_{(n,k)} := \{\mathbf{C} \mid \mathbf{C}_i = P(i) \text{ for some } k-1 \text{ degree polynomial } P\}.$$

Such codes are linear with the Vandermonde matrix representing one encoding matrix. An (n, k) Reed-Solomon code has the following features:

1. Let $\mathbf{C}_1, \dots, \mathbf{C}_n \in \text{RS}_{n,k}$ and let $\mathbf{C}'_1, \dots, \mathbf{C}'_n$ be some value. Define

$$\begin{aligned} k_{\text{correct}} &= |\{i \mid \mathbf{C}_i = \mathbf{C}'_i\}|. \\ k_{\text{error}} &= |\{i \mid \mathbf{C}_i \neq \mathbf{C}'_i \wedge \mathbf{C}'_i \neq \perp\}|. \\ e_{\text{erase}} &= |\{i \mid \mathbf{C}'_i = \perp\}|. \end{aligned}$$

where $k_{\text{correct}} + k_{\text{error}} + e_{\text{erase}} = n$. There exists an efficient procedure $\text{Decode}(\mathbf{C}'_1, \dots, \mathbf{C}'_n) = \mathbf{C}_1, \dots, \mathbf{C}_n$ when $2k_{\text{error}} + e_{\text{erase}} < n - k$. The Berlekamp-Welch algorithm is one such procedure. We assume that $\text{Decode}_{\text{RS}}$ outputs $\perp$ if $e_{\text{erase}} > n - k$.

2. For any $\ell \leq k$, $\text{Agree} = (i_1, y_{i_1}, \dots, i_\ell, y_{i_\ell})$ there exists a nonempty set RS_{Agree} such that for all $\mathbf{C} \in \text{RS}_{\text{Agree}}$, $\mathbf{C}_{i_j} = y_{i_j}$ for $j = 1, \dots, \ell$. (This is because all $k \times k$ minors of the Vandermonde matrix are full rank.)
3. One can efficiently sample from RS_{Agree} using an algorithm denoted as Inv_{RS} . Inv_{RS} takes the inverse of the square Vandermonde matrix corresponding to $i_1, \dots, i_\ell$ multiplied by $y_{i_1}, \dots, y_{i_\ell}$ to define the first ℓ components of the polynomial coefficients, the remaining $k - \ell$ coefficients are chosen uniformly.

Definition 3. A map $\text{Map} = (\text{insert}, \text{retrieve})$ is a pair of algorithms where,

1. $\text{Map.insert}(L, R)$: Adds (L, R) where L is the key and R is its assigned value.
2. $\text{Map.retrieve}(L)$: Given L , it returns the last assigned value R or $\perp$ if no value has been assigned.

We assume that values L and R are both binary strings of a fixed length. Looking ahead, values R will be from a field $\mathbb{F}_p$ we assume that $\log p$ is at most the supported length of the map. In a multimap, denoted as MM , $\text{MM.retrieve}(L)$ returns all previously assigned values R_i .

2.1 Approximate Proximity Search

We now turn to defining our cryptographic goal. We begin with the notion of a well-spread database which captures the intuition that its records are far apart.

Definition 4 (Well-spread database). For parameters $c > 1, t \in \mathbb{Z}^+$. For some value $y \in \{0, 1\}^\gamma$ and $\mathcal{DB} \in \{0, 1\}^{\gamma \times M}$, define

$$\begin{aligned}\text{Close}(y, \mathcal{DB}) &= \{x_i \mid x_i \in \mathcal{DB} \ \& \ \mathcal{D}(x_i, y) \leq t\}, \\ \text{Far}(y, \mathcal{DB}) &= \{x_i \mid x_i \in \mathcal{DB} \ \& \ \mathcal{D}(x_i, y) \geq ct\}.\end{aligned}$$

A database $\mathcal{DB} \in \{0, 1\}^{\gamma \times M}$ is said to be (c, t) -well-spread if

$$\forall y \in \{0, 1\}^\gamma, |\text{Far}(y, \mathcal{DB})| \geq |\mathcal{DB}| - 1.$$

This implies that $\forall y \in \{0, 1\}^\gamma, |\text{Close}(y, \mathcal{DB})| \leq 1$.

As discussed in the Introduction, Definition 4 is a strong condition useful for analysis. It is not satisfied by biometric data, see histograms for the IITD dataset in Figure 3(a). However, accuracy measurements in Section 5 consider real data.

Definition 5 (Approximate Proximity Search Scheme). Let $\text{APSS} = (\text{Init}, \text{Setup}, \text{Search})$. For $c > 1, t \in \mathbb{Z}^+$ APSS is a $(t, c, q, \delta_{\text{Far}}, \delta_{\text{Close}})$ -approximate proximity search scheme if for all (c, t) -well-spread $\mathcal{DB} \in \{0, 1\}^{\gamma \times M}$, $y_1, \dots, y_q \in \{0, 1\}^\gamma$, define

$$\begin{aligned}\begin{pmatrix} sk, pp \\ pp \end{pmatrix} &\leftarrow \text{APSS.Init} \begin{pmatrix} 1^\lambda \\ 1^\lambda \end{pmatrix}, \\ \begin{pmatrix} \perp \\ \mathcal{I}_0 \end{pmatrix} &\leftarrow \text{APSS.Setup} \begin{pmatrix} \mathcal{DB}, sk \\ pp \end{pmatrix}, \\ \begin{pmatrix} J_i \\ \mathcal{I}_i \end{pmatrix} &\leftarrow \text{APSS.Search} \begin{pmatrix} y_i, sk \\ \mathcal{I}_{i-1}, pp \end{pmatrix}.\end{aligned}$$

Then, for Far & Close defined as in Definition 4, it is true that, $\forall i, 1 \leq i \leq q$,

$$\begin{aligned}\Pr[\text{Far}(y_i, \mathcal{DB}) \cap J_i = \emptyset] &\geq 1 - \delta_{\text{Far}}, \\ \Pr[\text{Close}(y_i, \mathcal{DB}) \subseteq J_i] &\geq 1 - \delta_{\text{Close}}.\end{aligned}$$

Definition 6 (Adaptive Security for Search Protocol). Let $\text{SSE} = (\text{Init}, \text{Setup}, \text{Search})$ be a triple of algorithms with associated leakage functions $(\mathcal{L}^{\text{Setup}}, \mathcal{L}^{\text{Search}})$. Let λ be a security parameter.

For an adversary $\mathcal{A}$ and simulator $\mathcal{S}$ define $\text{Exp}_{\text{SSE}, \mathcal{A}}(\cdot)$ and $\text{Exp}_{\mathcal{S}, \mathcal{A}}(\mathcal{L} = (\mathcal{L}^{\text{Setup}}, \mathcal{L}^{\text{Search}}))$ as in Figure 1. We say SSE is semantically secure in the adaptive setting if for all PPT $\mathcal{A}$, there exists a PPT simulator $\mathcal{S}$ such that

$$|\Pr[\text{Exp}_{\text{SSE}, \mathcal{A}}(\cdot) = 1] - \Pr[\text{Exp}_{\mathcal{S}, \mathcal{A}}(\mathcal{L}^{\text{Setup}}, \mathcal{L}^{\text{Search}}) = 1]| \leq \text{negl}(\lambda).$$

We use Definition 6 for maps, multimaps, and approximate proximity schemes, which we denote at Map , MM and APSS respectively. We consider the following leakage functions:

1. $\mathcal{L}_{\text{Size}}^{\text{Setup}}$ which leaks the size of the created $\mathcal{DB}$. For the case of a map this leaks the number of (keyword, value) pairs inserted. Size is often padded to a power of 2 [HCD⁺25, GKL⁺20].
2. $\mathcal{L}_0^{\text{Search}}$, leaks the occurrence of a query [BIPW17, BT21].
3. $\mathcal{L}_{\text{AccPatt}}^{\text{Search}}$ which leaks identifiers returned with a query [SWP00, CGKO06]. These identifiers are consistent across queries.
4. $\mathcal{L}_{\text{QueryEq}}^{\text{Search}}$ which leaks when queries repeat in a sequence [LZWaT14, OK21].

Experiment $\text{Exp}_{\text{SSE}, \mathcal{A}}(\cdot)$:	Experiment $\text{Exp}_{\mathcal{S}, \mathcal{A}}(\mathcal{L}^{\text{Setup}}, \mathcal{L}^{\text{Search}})$:
1. $\begin{pmatrix} \text{sk} \\ \text{pp} \end{pmatrix} \leftarrow \text{SSE.Init} \begin{pmatrix} 1^\lambda \\ 1^\lambda \end{pmatrix}$. Let ts_{Init} be the server's view. 2. $D \leftarrow \mathcal{A}(\text{ts}_{\text{Init}})$. 3. $\begin{pmatrix} \perp \\ \mathcal{I}_0 \end{pmatrix} \leftarrow \text{SSE.Setup} \begin{pmatrix} D, \text{sk} \\ \text{pp} \end{pmatrix}$. Let ts_0 be the server's view. 4. For $i = 1$ to q: (a) $y_i \leftarrow \mathcal{A}(\text{ts}_{i-1})$. (b) $\begin{pmatrix} J_i \\ \mathcal{I}_i \end{pmatrix} \leftarrow \text{SSE.Search} \begin{pmatrix} y_i, \text{sk} \\ \mathcal{I}_{i-1}, \text{pp} \end{pmatrix}$. Let ts_i be the server's view. 5. Output $b \leftarrow \mathcal{A}(\text{ts}_q)$.	1. $\text{ts}_{\text{Init}} \leftarrow \mathcal{S}(1^\lambda)$. 2. $D \leftarrow \mathcal{A}(\text{ts}_{\text{Init}})$. 3. $\text{ts}_0 \leftarrow \mathcal{S}(\mathcal{L}^{\text{Setup}}(D))$. 4. For $i = 1$ to q: (a) $y_i \leftarrow \mathcal{A}(tk_{i-1})$. (b) $\text{ts}_i \leftarrow \mathcal{S}(\mathcal{L}^{\text{Search}}(y_i))$. 5. Output $b \leftarrow \mathcal{A}(\text{ts}_q)$.

Figure 1: Adaptive Experiments for search protocols and Adversary interacting with the Simulator in the ideal world using $\mathcal{L}$. Both $\mathcal{A}$ and $\mathcal{S}$ keep state between stages but this state is omitted for notational clarity.

During our construction we make n calls to the underlying map, in the case of $\mathcal{L}_0^{\text{Search}}$ this creates a straightforward leakage function as there are n calls to that map. Below we define two modifications of the above leakage functions. These leakage functions when one uses multiple LSHs in conjunction with a map. That is, they apply for the baseline or **ProxCode** system. They are a function of making multiple calls to the underlying map [KIK12, BT21, HCD⁺25]. For a query y and an integer n , we consider subqueries of the form $y_1, \dots, y_n$.

1. $\mathcal{L}_{\text{SubAccPatt}}^{\text{Search}}$ for an integer n , for each returned identifier ι leaks the pair (i, ι) of each subquery y_i that caused the identifier ι to be returned where $1 \leq i \leq n$.
2. $\mathcal{L}_{\text{SubQueryEq}}^{\text{Search}}$ for an integer n , leaks query equality over subqueries.

3 Baseline Construction

The goal of searching for a value y is to retrieve the set **Close** without receiving any indices in **Far**. We informally present the baseline LSH scheme, as described in prior work [KIK12, FWG⁺16, WYLH14, LPW⁺20, BT21, HCD⁺25], to introduce the relevant accuracy parameters. Let $\text{LSH}_1, \dots, \text{LSH}_n$ be a sampled set of LSHs and treat a as relevant for a value y if they agree on a single LSH value. The output is the set for $1 \leq j \leq n$:

$$\{x_i | \{(j, \text{LSH}_j(x_i))\} \cap \{(j, \text{LSH}_j(y))\} \neq \emptyset\}.$$

Construction 1 (LSH & Multimap based APSS). *Let t be a distance parameter, $c > 1$ and let $\mathcal{DB} \in \{0, 1\}^{\gamma \times M}$. Let LSH be a $(t, ct, \epsilon_t, \epsilon_t)$ -LSH family. Let MM be a multimap. Define $\text{APSS} = (\text{APSS.Init}, \text{APSS.Setup}, \text{APSS.Search})$ as in Algorithm 1. Then following notation from Definition 5, for all $y_1, \dots, y_q$, and all well-spread $\mathcal{DB}$, $\forall i$*

$$\begin{aligned} \Pr[\text{Far}(y_i, \mathcal{DB}) \cap J_i = \emptyset] &\geq 1 - \delta_{\text{Far}}, \\ \Pr[\text{Close}(y_i, \mathcal{DB}) \subseteq J_i] &\geq 1 - \delta_{\text{Close}}, \end{aligned}$$

$\text{Init} \begin{pmatrix} 1^\lambda \\ 1^\lambda \end{pmatrix} = \text{MM.Init} \begin{pmatrix} 1^\lambda \\ 1^\lambda \end{pmatrix}$
$\text{APSS.Setup}_n \left(\begin{array}{c} \mathcal{DB} = (x_1, \dots, x_M), \text{sk} \\ \text{pp} \end{array} \right):$
1. Sample n LSHs $\text{LSH}_1, \dots, \text{LSH}_n \leftarrow \text{LSH}$. 2. Set $\mathcal{DB}_{\text{MM}} = \{(j, \text{LSH}_j(x_i)), x_i\}_{j=1, \dots, M, i=1, \dots, n}$. 3. Execute $\begin{pmatrix} \perp \\ \mathcal{I}_0 \end{pmatrix} \leftarrow \text{MM.Setup}_n \left(\begin{array}{c} \mathcal{DB}_{\text{MM}}, \text{sk} \\ \text{pp} \end{array} \right)$. 4. Output $(\text{LSH}_1, \dots, \text{LSH}_n)$ to Client.
$\text{APSS.Search}_n \left(\begin{array}{c} y_i, \text{LSH}_1, \dots, \text{LSH}_n \\ \mathcal{I}_{(i-1) \cdot n} \end{array} \right):$
1. For $j = 1$ to n, compute $\begin{pmatrix} x_j \\ \mathcal{I}_{(i-1) \cdot n + j} \end{pmatrix} \leftarrow \text{MM.Search}_n \left(\begin{array}{c} (j, \text{LSH}_j(y)) \\ \mathcal{I}_{(i-1) \cdot n + (j-1)} \end{array} \right)$. 2. Output $J_i = \cup_{j=1}^n x_j$.

Algorithm 1: Baseline construction of APSS from LSH and MM. All procedures from perspective of Client with calls to underlying interactive protocols.

For

$$\begin{aligned} \delta_{\text{Far}} &= 1 - (1 - \epsilon_{\text{f}})^{nM}, \\ \delta_{\text{Close}} &= (1 - \epsilon_{\text{t}})^n. \end{aligned}$$

That is, APSS is a $(t, c, \delta_{\text{Far}}, \delta_{\text{Close}})$ -approximate proximity search scheme.

δ_{Close} exactly corresponds with TAR for a well-spread database. However, δ_{Far} controls the overall probability of a false accept and is a much stronger condition than controlling the FAR. Below we analyze the FAR of the baseline scheme for random data where each record in the database has exactly ϵ_{f} probability of matching an LSH and each query that is a noisy version of a stored x_i has probability exactly ϵ_{t} of colliding LSH with the stored reading of the biometric. We use report FAR as δ_{Far} for consistency with ProxCode, which is presented shortly. This means we are comparing ProxCode against a baseline scheme with a weaker correctness guarantee.

As described in the Introduction, there are three main issues with Construction 1:

1. The use of a multimap. Constructing oblivious multimaps is a difficult prospect (see discussion in [KMO18, GKM21, GPP23, AG22, RW23]).
2. The use of a disjunctive query requires ϵ_{f} to be very small and n to be very large to support reasonable $\delta_{\text{Close}}, \delta_{\text{Far}}$. Table 5 highlights this comparison.
3. In the three party searchable encryption scenario, unintended biometrics are (occasionally) leaked to clients.

3.1 Finding n for baseline construction

For fixed ϵ_t, ϵ_f it suffices to set

$$\frac{\log(\delta_{\text{close}})}{\log(1 - \epsilon_t)} \leq n \leq \frac{\log(\delta_{\text{far}})}{M \log(1 - \epsilon_f)}. \quad (1)$$

In particular, in the setting when $\delta_{\text{close}} \approx \delta_{\text{far}}$ and for small ϵ_t, ϵ_f where $\log(1 - x) \approx -x$ for n to exist in Equation 1 it must be the case that $\epsilon_t \geq M\epsilon_f$.

In the case when the LSH is an extended LSH with underlying error rates of ϵ'_t, ϵ'_f with α concatenated copies and $\epsilon'_t > \epsilon'_f$ then setting

$$\alpha \geq \frac{\log(M)}{\log(\epsilon'_t/\epsilon'_f)}. \quad (2)$$

suffices for

$$\left(\frac{\epsilon'_t}{\epsilon'_f}\right)^\alpha = \frac{\epsilon_t}{\epsilon_f} \geq M.$$

This means that

$$n \approx \frac{M \log(\delta_{\text{close}})}{\log(\delta_{\text{far}})}. \quad (3)$$

4 ProxCode

This section formally introduces **ProxCode**, and gives the proof that it is secure and accurate under the well-spread condition (Definition 4). This condition is not assumed in our evaluation in Section 5.

Our construction combines LSHs and secure map. Instead of assigning LSH outputs with records, we assign them with points on a random polynomial whose intercept is the record. Later, multiple shares are collected and decoded. **Search** only reveals a matching record x_i when there is *enough* LSH matches.

To handle LSH collisions, when $z = \text{LSH}_\beta(x_i) = \text{LSH}_\beta(x_j)$, we constrain the two polynomials to have the same value at position β . That is, that $\mathbf{c}_{i,\beta} = \mathbf{c}_{j,\beta}$. We can do this without impacting either $\mathbf{c}_{i,1}$ or $\mathbf{c}_{k,1}$ as long as this occurs in no more than $k - 1$ positions for each codeword. We present this scheme formally in Algorithm 2 and Construction 2.

As mentioned in the Introduction, the goal can be retrieval of the indices, i , or the actual values, x_i (see discussion in Gui et al. [GPPW24, GPP23]). Usually in encrypted search, one focuses on building an index data structure with the actual records being obtained through a second oblivious structure. Our system works equally well in both settings, assuming the map can hold entire records (as long as they are distinct) since our encoding technique does not increase the size of values inserted in the map (beyond the additional space to encode them in a field). A separate lookup of the value x_i from i often has leakage, so we associate x_i to prevent the second lookup.

Construction 2. Let t be a distance and let $c > 1$ be a distance parameter. Let $\mathcal{DB} \in \{0, 1\}^{\gamma \times M}$ be a (c, t) -well-spread database. Let $n \in \mathbb{Z}^+$ and $\mu, k \in \mathbb{Z}^+$ such that $\mu \leq k < n$.

1. Let **LSH** be a family of $(t, ct, \epsilon_t, \epsilon_f)$ -LSHs with domain of $\{0, 1\}^r$.
2. Let p be a prime power such that $p \geq M$. Let $\text{RS}_{(n+1, k)}$ be a Reed-Solomon code with associated algorithms $\text{Decode}_{\text{RS}}, \text{Inv}_{\text{RS}}$.
3. Let $\text{Map} = (\text{Map.insert}, \text{Map.retrieve})$ be a map.

Define **APSS** as in Algorithm 2.

We provide some intuition for the scheme before presenting our formal results. There are two main ideas in Construction 2:

Polynomial evaluations in the Map. First, we replace x_i as the value inserted into the map with a polynomial whose intercept is x_i . That is, $\mathbf{c}_{i,1} = x_i$. The idea behind this change is that if one can reconstruct $\mathbf{c}_i$ then one can easily recover the value x_i . We then insert pairs $((j, \text{LSH}_j(x_i)), \mathbf{c}_{i,j})$ into the map.

$\text{Init} \begin{pmatrix} 1^\lambda \\ 1^\lambda \end{pmatrix} = \text{Map.Init} \begin{pmatrix} 1^\lambda \\ 1^\lambda \end{pmatrix}$ $\begin{pmatrix} \text{LSH}_1, \dots, \text{LSH}_n \\ \mathcal{I} \end{pmatrix} \leftarrow \text{Setup} \begin{pmatrix} \mathcal{DB}, \text{sk} \\ \text{pp} \end{pmatrix}:$ 1. Sample $\text{LSH}_1, \dots, \text{LSH}_n \leftarrow \text{LSH}$. Define $L \in (\{0, 1\}^r)^{M \times n}$ where $L_{i,j} = \text{LSH}_j(x_i)$. 2. Define $\text{Eq} \in [M]^{M \times n}$ where $\text{Eq}_{i,j} = \arg \min_{i' < i} (L_{i',j} = L_{i,j})$ where $\text{Eq}_{i,j} = 0$ if no such i' exists. 3. If there exists a row of Eq with more than $k-1$ nonzero coordinates go to Step 1 (up to ℓ times, then output $\perp$). 4. Initialize $\mathbf{C} \in (\mathbb{F}_p \cup \perp)^{M \times (n+1)} = \perp^{M \times (n+1)}$ 5. For $i = 1, \dots, M$: (a) $\mathbf{C}_{i,1} = x_i$. (b) For $j = 1, \dots, n$, if $\text{Eq}_{i,j} \neq 0$, set $\mathbf{C}_{i,j+1} = \mathbf{C}_{\text{Eq}_{i,j},j+1}$. (c) $\mathbf{C}_i = \text{Inv}_{\text{RS}}(\text{NEmpty}(\mathbf{C}_i))$. NEmpty outputs the indices and values of positions that are not $\perp$. 6. $\mathcal{DB} = (j \ \text{LSH}_j(x_i), \mathbf{C}_{i,j+1})$ for $i = 1, \dots, M, j = 1, \dots, n$. 7. $\begin{pmatrix} \perp \\ \mathcal{I}_0 \end{pmatrix} \leftarrow \text{Map.Setup} \begin{pmatrix} \mathcal{DB}, \text{sk} \\ \text{pp} \end{pmatrix}$.	$\text{Search} \begin{pmatrix} y, \text{LSH}_1, \dots, \text{LSH}_n, \text{sk} \\ \mathcal{I}_{(i-1) \cdot n}, \text{pp} \end{pmatrix}:$ 1. Compute $L_j = \text{LSH}_j(y)$ for all $j = 1, \dots, n$. 2. Initialize $\mathbf{e}_{\text{erase}} = n$. 3. For $j = 1, \dots, n$, (a) Client retrieves $\begin{pmatrix} \mathbf{c}_{j+1} \\ \mathcal{I}_{(i-1) \cdot n + j} \end{pmatrix} = \text{Map.Search} \begin{pmatrix} (j, L_j), \text{sk} \\ \mathcal{I}_{(i-1) \cdot n + (j-1)}, \text{pp} \end{pmatrix}$. (b) If $\mathbf{c}_{j+1} \neq \perp$, $\mathbf{e}_{\text{erase}} := \mathbf{e}_{\text{erase}} - 1$. 4. If $\mathbf{e}_{\text{erase}} > n - k$ output $\perp$. 5. Compute $\mathbf{c}_1, \dots, \mathbf{c}_k \leftarrow \text{Decode}_{\text{RS}}(\perp \ \mathbf{c}_2 \ \dots \ \mathbf{c}_n)$, output $\mathbf{c}_1$
---	---

Algorithm 2: ProxCode: APSS from maps and linear codes. Procedures are run by Client unless calling an underlying interactive protocol.

Align codewords with LSH collisions. We add a preprocessing step so that codewords are chosen in a correlated manner. We precompute Eq , the set of all LSH collisions in the database. If two values x_i, x_k share some $\text{LSH}_\beta(x_i) = \text{LSH}_\beta(x_k)$ then we fix $\mathbf{c}_{i,\beta} = \mathbf{c}_{k,\beta}$. We rely on the fact that one can interpolate a degree $k-1$ polynomial for any k points to ensure this is possible. Theorem 1 bounds the probability that such sampling cannot complete over the choice of LSH_i s. Importantly, this probability holds for all well-spread $\mathcal{DB}$ and does not depend on the chosen codewords. We check this condition by examining Eq .

Once Setup is complete, there is now a one-to-one correspondence between the LSH outputs and the codeword symbols. Let $x_i \in \mathcal{DB}$, if one searched for the value x_i one would retrieve $\mathbf{c}_{i,2}, \dots, \mathbf{c}_{i,n+1}$ which determines $\mathbf{c}_{i,1}$ and allow retrieval. If one searches for a value y then the returned values will be a mix of different codewords and $\perp$ if nothing in the database matched the LSH value. We first consider the correctness of this scheme deferring security until Section 4.2.

4.1 Correctness

Theorem 1. *Let $c, c_1, c_2 > 0$ be constants. Let LSH be a family of $(t, ct, \epsilon_t, \epsilon_f)$ -locality sensitive hashes. Let $\mathcal{DB}$ be a (c, t) -well-spread database where $|\mathcal{DB}| = M$. Let $n \in \mathbb{Z}^+, k \in \mathbb{Z}^+$ be parameters. Suppose the*

following are true:

$$\epsilon_t > \frac{2k}{(1-c_2)n}, \quad (4)$$

$$\epsilon_f < \frac{k}{Mn(1+c_1)}, \quad (5)$$

and define

$$\begin{aligned} \forall i \leq M, \delta_{\text{Far}_i} &= \exp\left(\frac{-c_1^2}{2+c_1} \cdot \epsilon_f \cdot n \cdot (i-1)\right), \\ \delta_{\text{Far}} &= \delta_{\text{Far}_M} = \exp\left(\frac{-c_1^2}{2+c_1} \cdot \epsilon_f \cdot n \cdot (M-1)\right), \\ \delta_{\text{Close}} &= \exp\left(\frac{-c_2^2 \epsilon_t n}{2}\right) + \delta_{\text{Far}_{M-1}}. \end{aligned}$$

Construction 2 instantiated with $\text{RS}_{(n+1,k)}$ and n LSHs from LSH is an $(t, c, \delta_{\text{Far}}, \delta_{\text{Close}})$ -APSS. Furthermore,

$$\Pr[\text{Setup outputs } \perp] \leq \left(1 - \prod_{i=2}^M (1 - \delta_{\text{Far},i})\right)^\ell \leq \left(1 - (1 - \delta_{\text{Far}})^{M-1}\right)^\ell$$

Theorem 1 is proved through Lemmas 1, 2, and 3 which focus on the number of LSH matches between close records, far records, and the ability of setup to complete. Roughly, each of these lemmas is proved using a Chernoff bound since LSH outputs are independent (if data is fixed before sampling). The constants c_1, c_2 represent the constant of the Binomial deviating from its expectation.

Lemma 1. *Let all parameters be as in Theorem 1. Define*

$$\text{Match}_{j,x,x^*} = \begin{cases} 1 & \text{LSH}_j(x) = \text{LSH}_j(x^*) \\ 0 & \text{otherwise} \end{cases}.$$

And define $\text{Match}_{x,x^*} = \sum_{j=1}^n \text{Match}_{j,x,x^*}$. If $\mathcal{D}(x, x^*) \leq t$ then

$$\Pr[\text{Match}_{x,x^*} \leq 2k] < \exp\left(\frac{-c_2^2}{2} \cdot \epsilon_t \cdot n\right).$$

Proof. Let x, x^* be two values where $\mathcal{D}(x, x^*) \leq t$. One has

$$\forall j, \text{Exp}[\text{Match}_{j,x,x^*}] > \frac{2k}{(1-c_2)n}$$

by Equation 4. By independence of the LSHs, Match_{x,x^*} is bounded below by a $(n, \frac{2k}{(1-c_2)n})$ binomial distribution with $\text{Exp}[\text{Match}_{x,x^*}] = \frac{2k}{(1-c_2)}$. Then by a standard Chernoff bound, it is true that

$$\begin{aligned} \Pr[\text{Match}_{x,x^*} \leq 2k] &= \Pr[\text{Match}_{x,x^*} < (1-c_2)\text{Exp}[\text{Match}_{x,x^*}]] \\ &< \exp\left(\frac{-c_2^2}{2} \cdot \epsilon_t \cdot n\right). \end{aligned}$$

This completes the proof of Lemma 1. □

Lemma 2. Let all parameters be as in Theorem 1. Define random variable $\text{Match}_{j,\mathcal{DB},x}$ as follows for $j \in \{1, \dots, n\}$:

$$\text{Match}_{j,\mathcal{DB},x} = |\{x_i \in \mathcal{DB} \mid \text{LSH}_j(x_i) = \text{LSH}_j(x)\}|.$$

and $\text{Match}_{\mathcal{DB},x} = \sum_{j=1}^n \text{Match}_{j,\mathcal{DB},x}$, denoting the number of LSH's where there exists some collision between the value x' and some record in the $\mathcal{DB}$. For all x such that $\forall x_i \in \mathcal{DB}$ it is true that $\mathcal{D}(x, x_i) \geq ct$ it is true that

$$\Pr[\text{Match}_{\mathcal{DB},x} \geq k] \leq \exp\left(\frac{-c_1^2}{2+c_1} \cdot \epsilon_f \cdot n \cdot M\right).$$

Proof. For each pair x, x' such that $\mathcal{D}(x, x') \geq ct$ it is true that

$$\Pr_{\text{LSH} \leftarrow \text{H}_{1\text{sh}}}[\text{LSH}(x) = \text{LSH}(x')] \leq \epsilon_f.$$

This means that $\text{Match}_{\mathcal{DB},x}$ is bounded above by a (nM, ϵ_f) binomial distribution. By a standard Chernoff bound one has

$$\begin{aligned} \Pr[\text{Match}_{\mathcal{DB},x} > k] &= \\ \Pr[\text{Match}_{\mathcal{DB},x} > (1+c_1)\text{Exp}[\text{Match}_{\mathcal{DB},x}]] &\leq \\ \exp\left(\frac{-c_1^2}{2+c_1} \cdot \epsilon_f \cdot n \cdot M\right) \end{aligned}$$

This completes the proof of Lemma 2. □

Lemma 3. Let all parameters be as in Lemma 2 and Theorem 1 letting

$$\delta_{\text{Far},i} = \exp\left(\frac{-c_1^2}{2+c_1} \cdot \epsilon_f \cdot n \cdot (i-1)\right).$$

Then the probability that **Setup** outputs $\perp$ is at most

$$\Pr[\text{Setup outputs } \perp] \leq \left(1 - \prod_{i=2}^M (1 - \delta_{\text{Far},i})\right)^\ell \quad (6)$$

Proof. Let $(x_1, \dots, x_M) = \mathcal{DB}$. For all $x_i \in \mathcal{DB}$ define $\mathcal{DB}_{x_i} = x_1, \dots, x_{i-1}$. By the (c, t) -well-spread condition of $\mathcal{DB}$ and Lemma 2 it is true that

$$\Pr[\text{Match}_{\mathcal{DB}_{x_i}, x_i} \geq k] \leq \delta_{\text{Far},i}.$$

Setup succeeds in an iteration if it is true for all $x_i \in \mathcal{DB}$ that $\text{Match}_{\mathcal{DB}_{x_i}, x_i} < k$. Let 1_{x_i} be an indicator random variable where $1_{x_i} = 1$ if $\text{Match}_{\mathcal{DB}_{x_i}, x_i} < k$. Then

$$\Pr\left[\sum_{i=2}^M 1_{x_i} = 0\right] \geq \prod_{i=2}^M (1 - \Pr[\text{Match}_{\mathcal{DB}_{x_i}, x_i} \geq k]).$$

So the chance that an iteration of setup fails is at most

$$\Pr\left[\sum_{x_i} 1_{x_i} > 0\right] \leq 1 - \prod_{i=2}^M (1 - \Pr[\text{Match}_{\mathcal{DB}_{x_i}, x_i} \geq k]).$$

The chance that all ℓ iterations fail is then at most

$$\left(1 - \prod_{i=2}^M (1 - \Pr[\text{Match}_{\mathcal{DB}_{x_i}, x_i} \geq k])\right)^\ell.$$

This completes the proof of Lemma 3. □

Proof of Theorem 1. There are three parts to proving Theorem 1 that setup completes with high probability, that the close item is included in the result set and that far items are not included in the result set. The probability of setup completing follows directly from Lemma 3.

Close Item in Result Set. Let x be the search term where x is close to at most one item in $\mathcal{DB}$ denoted as x_i with corresponding codeword $\mathbf{c}_i$. That is, $\mathcal{D}(x, x_i) \leq t$. If such a x_i exists, its uniqueness is guaranteed by Definition 4. Let $\mathbf{c}_i$ denote the corresponding codeword. Define the following parameters:

$$\delta_{\text{close},1} = \exp\left(\frac{-c_2^2}{2} \cdot \epsilon_t \cdot n\right),$$

$$\delta_{\text{close},2} = \delta_{\text{Far},(M-1)}.$$

Let $\mathbf{c}'$ denote the recovered symbols (including symbols that are $\perp$). By Lemma 1 there are at least $2k$ symbols from $\mathbf{c}_i$ with probability $1 - \delta_{\text{close},1}$. By Lemma 2 there are at most k_{error} symbols from the other LSH values $M - 1$. By union bound, both of these conditions hold with probability $1 - (\delta_{\text{close},1} + \delta_{\text{close},2})$. Conditioned on both of these events occurring $\text{Decode}_{\text{RS}}$ outputs $\mathbf{c}_i$ with probability 1.

Far Items not in Result Set. By Lemma 2 the probability that $\mathbf{c}'$ has more than $k - 1$ symbols other than $\perp$ is at most δ_{Far} .

This completes the proof of Theorem 1. $\square$

4.2 Security and Leakage

This section shows that when the **Map** in Construction 2 is an appropriate encrypted map, one achieves a secure APSS. The proofs in this section are straightforward. We consider two commonly used leakage patterns in secure maps: 1) the zero-leakage setting where the server learns the size of the dataset M and when a query occurs, such as [BIPW17,BT21] and 2) access and search pattern where the server learns the identifiers associated with each query response and whether the queries have been repeated [SWP00,CGKO06]. Of course, if one uses a zero-leakage map [WNL⁺14,BT21], the resulting APSS is zero-leakage as well (treating n as a public system parameter). Since each query of the APSS translates to n queries to the underlying map, we additionally leak when subqueries repeat and the subquery associated with a returned identifier.

Lemma 4. *Let λ be a security parameter. Let $\text{Map} = (\text{Map.Setup}, \text{Map.Search})$ be a map that is secure according to Definition 6 for*

$$\mathcal{L}_{\text{Map}} = \mathcal{L}^{\text{Setup}} = |\text{Map}|,$$

$$\mathcal{L}^{\text{Search}} = (\mathcal{L}_{\text{Acc Patt}}^{\text{Search}}, \mathcal{L}_{\text{Query Eq}}^{\text{Search}}).$$

Then the APSS = (APSS.Init, APSS.Setup, APSS.Search) scheme defined in Construction 2 is secure according to Definition 6 for

$$\mathcal{L}^{\text{Setup}} = n \cdot |\text{Map}|,$$

$$\mathcal{L}^{\text{Search}} = (\mathcal{L}_{\text{Sub Acc Patt}}^{\text{Search}}, \mathcal{L}_{\text{Sub Query Eq}}^{\text{Search}}).$$

Proof. Let $\mathcal{A}_{\text{APSS}}$ denote some PPT adversary for the APSS scheme. Our goal is to construct a $\mathcal{S}_{\text{APSS}}$. As noted in Figure 2 for any valid $\mathcal{A}_{\text{APSS}}$ adversary there exists some $\mathcal{A}_{\text{Map}}$ that is a valid **Map** adversary. Let $\mathcal{S}_{\text{Map}}$ be one such simulator for $\mathcal{A}_{\text{Map}}$. Note that setup leakage is the same in both settings. For the search leakage, $(\mathcal{L}_{\text{Sub Acc Patt}}^{\text{Search}}, \mathcal{L}_{\text{Sub Query Eq}}^{\text{Search}})$ this allows $\mathcal{S}_{\text{APSS}}$ to expand the q queries into qn subqueries which is the required leakage for $\mathcal{S}_{\text{Map}}$. Then

$$\begin{aligned} & |\Pr[\text{Exp}_{\text{APSS}, \mathcal{A}_{\text{APSS}}}(\cdot) = 1] - \\ & \Pr[\text{Exp}_{\mathcal{S}_{\text{APSS}}, \mathcal{A}_{\text{APSS}}}(\mathcal{L}_0^{\text{init}}, \mathcal{L}_{\text{Sub Query Eq, Sub Acc Patt}}^{\text{Search}}) = 1]| = \\ & |\Pr[\text{Exp}_{\text{Map}, \mathcal{A}_{\text{Map}}}(\cdot) = 1] - \\ & \Pr[\text{Exp}_{\mathcal{S}_{\text{Map}}, \mathcal{A}_{\text{Map}}}(\mathcal{L}_0^{\text{init}}, \mathcal{L}_{\text{Query Eq, Acc Patt}}^{\text{Search}}) = 1]| \end{aligned}$$

This completes the proof of Lemma 4. $\square$

$\mathcal{A}_{\text{Map}}.\text{Setup}(1^\lambda) :$	$\mathcal{A}_{\text{Map}}.\text{Search}(\vec{\text{LSH}}, L, \mathbb{C}) :$
1. Initialize $\mathcal{A}^{\text{APSS}}$ and receive $\mathcal{DB} \in (\{0, 1\}^\gamma)^M$.	1. Receive $q \in \{0, 1\}^\gamma$ from $\mathcal{A}^{\text{APSS}}$.
2. Run steps 1-6 of $\text{APSS}.\text{Setup}(\mathcal{DB})$ from algorithm 2 to receive vector $\vec{\text{LSH}}$ and matrices L and $\mathbb{C}$ as described in steps 1, 2 and 6 respectively. If Step 4 outputs $\perp$ output $\perp$.	2. Compute $q_1, \dots, q_n = \vec{\text{LSH}}_1(q), \dots, \vec{\text{LSH}}_n(q)$.
3. Output $\mathcal{DB}_{\text{Map}} = \{(L_{i,j}, \mathbb{C}_{i,j+1})\}_{j=1, \dots, n}^{i=1, \dots, M}$.	3. Output $q_1, \dots, q_n$.
	4. Receive tk and send to $\mathcal{A}^{\text{APSS}}$.

Figure 2: Construction of $\mathcal{A}_{\text{Map}}$ from $\mathcal{A}_{\text{APSS}}$.

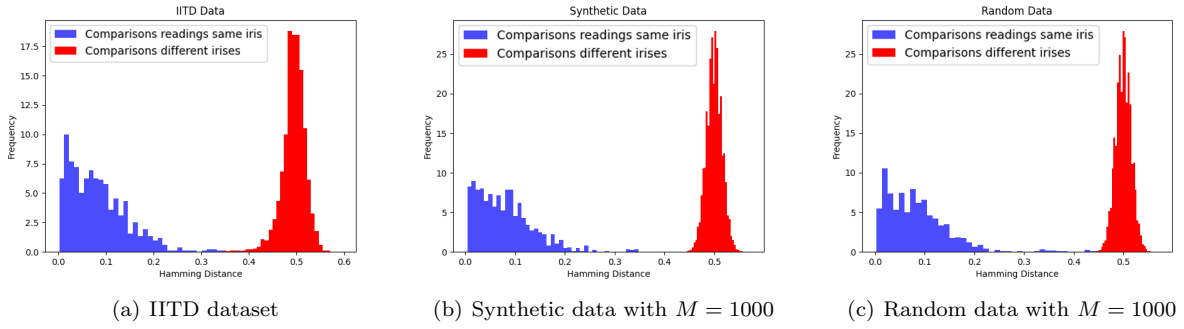


Figure 3: FHD histogram for real data from IITD dataset, synthetic data, and random data. Comparisons between readings of the same biometric are in blue. Comparisons between readings of different biometrics are in red. The x-axis differs.

4.3 Discussion

Handling Dynamic Data. Assuming a dynamic map, one can naturally handle new data x^* being added to the database by searching for x^* and retrieving codeword symbols that x^* 's codeword should be consistent with. Then one can sample the codeword (under the constraints described above) and add the missing codeword symbols to the corresponding maps. Handling data deletion and updates requires care; map values have information about multiple biometrics. One way to handle deletes is to maintain a counter with each value indicating how many records are using this value, this counter could be decremented with each delete. The leakage and efficiency of the above depends strongly on the underlying map. Further study is required.

5 Implementation and Accuracy

We implemented **ProxCode** using Python 3. Our implementation uses an implementation of the tree-based oblivious map of Wang et al. [WNL⁺14]. The source code can be found in (Github).

5.1 Used Data

We perform tests on iris, synthetic, and random data. Iris datasets are not available for large M .

Real Data The IITD dataset [KP10] consists of 224 persons and 2240 images. We process these data using a recent feature extractor used for LSH-based fuzzy extractors [SDF⁺25] and segmentation system of Ahmad and Fuller [AF19]. We use their regime of left irises for training and right irises for testing. After

removing unsuitable right irises without two readings, there are $M = 202$ right irises suitable for testing. We choose this feature extractor as it is designed to produce good features for extended LSHs.

Synthetic Data Since the existing Real datasets such as IITD are small, to test our construction on a larger dataset with a distribution closer to the real world than random, we used synthetic dataset. To generate an initial reading of irises we perform the following:

1. Measure the degrees of freedom and closest fit binomial distribution for the real data in the previous subsection. This is the standard mechanism for estimating the entropy of an iris after applying a feature extractor [Dau09].
2. This yielded a binomial with parameters of $n = 391, p = .4939$.
3. For each starting iris we sampled from a binomial with (n, p) as above.
4. Converted the output to 512 bit features by multiplying by $512/391$ and rounding.
5. This specified how many 1s to put in the feature vector.
6. We then randomly placed the chosen number of 1s in the feature vector.

For generating noise between readings of the same iris, we use the probability mass function for number of errors from our real-data and randomly place errors.

Random Data Random data uses the same error distribution as synthetic data but all feature vectors are uniformly distributed. Histograms for all three datatypes are shown in Figure 3.

5.2 Time Efficiency

Our savings in n will translate to any encrypted map, see discussion in the Introduction. For our cryptographic implementation, we use a tree-based oblivious map due to Wang et al. [WNL⁺14]. At a high level, the construction is a tree of oblivious arrays where one stores the current logical position of all children for a node in addition to the node value. This storage allows one to use non-recursive ORAM, as long as one always traverses the tree from the root to some predetermined depth. In our setting, we always traverse to a leaf node for each LSH subquery. One can traverse the tree using PathORAM [SDS⁺18] using depth + 1 rounds of communication.

Usually one queries the map on all n LSH values of the search term. Ha et al. [HCD⁺25] build a preprocessing step using private set intersection where one first finds out which terms exist in the map and then uses the map to search only a constant number of non-null terms. They find the cryptographic overhead of this first stage is small compared to overhead of the second stage.

We use κ to denote the required number of parallel accesses to ORAM that doesn't impact accuracy. Let $\max_{\text{error}}$ be the maximum number of errors observed in the query set. Then $\kappa = k + 2\max_{\text{error}} + 1$. By making κ queries in this way, one always retains enough codeword symbols to reconstruct the value without having to query all n terms. Often $\kappa/n \approx .1$ improving performance by an order of magnitude. Furthermore, κ is usually of the scale that queries could be processed in parallel by a modern server.

We consider two timing experiments below, how long to preprocess in ProxCode and timing for Search and Setup of the full implementation. These evaluations are on a Ryzen Threadripper PRO 7995WX with 384MB L3 cache and 768GB of DDR5 RAM.

Finding parameters We manually find the relevant n, k, α for each dataset. We choose a target TAR of .90 for a parameter set to be acceptable. We consider dataset sizes for synthetic and random of $M = \{1000, 2500, 5000, 10000, 25000\}$.

M	Dataset type	# Queries	n	k	α	Sample LSH	Eq. Matrix
202	IITD	202	100	4	17	.15	.051
1000	Synth	200	500	12	17	3.7	1.4
1000	Rand					3.6	1.5
2500	Synth	200	600	10	20	12.1	5.1
2500	Rand					11.8	5.1
5000	Synth	200	700	13	21	31.0	13.4
5000	Rand					30.9	12.9
10000	Synth	200	800	10	23	70.6	34.2
10000	Rand					71.0	34.2
25000	Synth	200	800	10	25	191	98.4
25000	Rand					190	95.0

Table 3: PreProcessing Times (in seconds) for Tested Concrete Parameters.

M	Dataset type	# q	n	k	α	TAR	Map K Size	Par Rnds	Crypto Time	Setup GB	O Reads	Search Time		
202	IITD	202	100	4	17	.97	20.2	16	13	.04	1.5	.76	1.4	9
1000	Synth	200	500	12	17	.96	500	19	418	1.1	9.5	3.0	2.5	27
1000	Rand					.91			421			3.0	2.4	27
2500	Synth	200	600	10	20	.94	1500	21	1430	3.3	12.6	4.11	2.9	23
2500	Rand					.92			1430			4.11	3.0	23
5000	Synth	200	700	13	21	.90	3500	22	3560	7.6	15.4	5.20	3.4	22
5000	Rand					.89			3500			5.32	3.2	22
10000	Synth	200	800	10	23	.90	8000	23	8600	17.4	18.4	6.82	3.5	17
10000	Rand					.86			8520			6.73	3.5	17
25000	Synth	200	800	10	25	.88	20000	25	23700	43.5	20.0	7.17	4.2	17
25000	Rand					.88			24300			7.53	5.1	19

Table 4: Cryptographic Efficiency results. Map sizes and number of Reads (number of ORAM accesses per search) are in thousands. Setup Time and Sequential Search Times are in seconds, Parallel Search time in ms. GB is 10^9 bytes.

ProxCode preprocessing time We first measure time to run Algorithm 2 ignoring the map, note this algorithm has complexity $\Theta(n \times M)$, assuming constant cost to evaluate each LSH. Results are reported in Table 3. On our largest tested parameter sets finding the relevant polynomials takes at most 5 minutes. Looking ahead, this time is dwarfed by the time to build the relevant oblivious map which on the same data set is nearly 7 hours. For all of our chosen parameters, **Setup** completes in a single iteration so we don't report the number of iterations.

Cryptographic Efficiency. In Table 4 we report the time to set up all the oblivious RAM arrays, the number of rounds, the number of ORAM reads, the overall search time, and the parallel search time which is the average (across queries) of the maximum time required for each subquery. The overall search time includes decoding the code and as such is more than n times the parallel search time. Network delay is not modeled. While setup time is in hours for our largest datasets, parallel search still requires $< 10ms$. On a relatively fast network with a 100ms RTT, overall query time would be roughly 2.5s. As we discuss above we can use the same filtering technique of Ha et al. [HCD⁺25] to only search κ times in the map and κ remains small enough that all requests could be reasonably handled by a modern server. We do note that our κ are much smaller than Ha et al. who use $\kappa = 72$ on synthetic data of size 5000 (they use δ for the number of searched items in the map).

5.3 Implications of ProxCode for Client Security

Throughout this work, we defined and considered the two party SSE setting for notational simplicity. Since **ProxCode** is primarily a preprocessing technique it naturally extends to a three party SSE setting. A three party SSE consider a data owner, server, and a client [FVK⁺15]. In this setting, in addition to limiting leakage to the server, the data owner wishes to limit unintended data learned by a client. The three party setting highlights the importance of an accurate system. To demonstrate the difference between the baseline and **ProxCode** we demonstrate the difference in information available between map outputs to the client. This information is available even if one uses a fully oblivious map.

- **Baseline:** For fixed value $n = 100$ we found the α for Baseline with comparable TAR. This value is $\alpha = 22$, producing a TAR of .923. For these values across the query set, this produced 4 false positives across the 202 queries. Each false positive represents a biometric of a non-relevant person incidentally exposed to the client.
- **ProxCode:** For parameters $\alpha = 17, n = 100, k = 8$ (first row in Table 4), we fixed a single value x_i and issued queries corresponding to the other irises in the dataset. We then measured how many values $\mathbf{C}_{i,j}$ for $2 \leq \mathbf{C}_{i,j} \leq n + 1$ are returned by some other query. That is, we measure how many “shares” of $\mathbf{C}_i$ are obtained after 201 queries for each of other irises. This produced the average of 1.4 shares across 201 queries, with STD of .66, and the max of 4 shares. No iris has at least k symbols returned after issuing 201 queries.

To summarize, in the Baseline system a client who issues a small number of queries has a small probability of learning some non-relevant iris. In **ProxCode** a client may be able to decode a single iris after 200 queries. Analysis assumes client queries come from irises in the dataset. One expects higher success with specifically crafted queries [ZKP16, ZWX⁺23].

6 Conclusion and Future Work

In this work, we consider approximate proximity searchable encryption in both zero and access pattern leakage settings. Our scheme allows use of a map and reduces leakage over the baseline scheme.

This work considers Reed-Solomon codes that correct arbitrary errors. However, observed errors are not arbitrary. Assume that the ϵ_t is tuned so that $k' > k$ LSH matches occur for a nearby value with good probability. The actual errors are defined by the following process:

1. Sample M codewords $\mathbf{c}_1, \dots, \mathbf{c}_M$ (under the collision constraint defined above).
2. Consider codeword $\mathbf{c}_i$ corresponding to a search for a value x^* that is close to x_i . Consider a fixed a symbol j . With probability at least $1 - \epsilon'_t$ the symbol $\mathbf{c}_{i,j}$ is correctly transmitted. Otherwise there are two cases:
 - (a) With probability at most $(M-1)\epsilon'_t$ is replaced by $\mathbf{c}_{i,k}$ for some $k \neq j$. Each of these replacements occur with probability at most ϵ'_t .
 - (b) Otherwise, the symbol is converted to $\perp$.

There are two aspects of the above error model: 1) errors come from symbols of other codewords, and 2) these codewords are not independently sampled.

It is an open problem to design codes that correct more of such errors than is possible in traditional error models. Furthermore, it seems possible to argue some independence and randomness of the errors (Shannon model [GRS22]) using the secret sharing properties of the code. We were not able to prove this or find a counterexample. The sticking point was the coupled sampling of the codewords. We also consider using list decoding for Reed Solomon codes. There are three reasons **ProxCode** could not work:

1. Setup doesn't complete,
2. There are too many errors to successfully decode, and

3. There were too few LSH matches between noisy readings, which creates too many erasures.

In our experience, items 1 and 3 were bigger issues when working with real data. This can be seen by the relatively small values of κ . For example, the largest number of errors in the random 25000 query set was only 4. We didn't find the need to implement more complex correction beyond Berlekamp Welch.

ProxCode has a dramatic impact on the efficiency of LSH based proximity searchable encryption. These improvements lead to major improvements in all aspects of system efficiency. We present an open-source prototype implementation with testing for real, synthetic, and random data.

Acknowledgements

The authors are grateful to the reviewers for their important comments in improving this work. The work of B.F. and M.R. is supported by NSF grants #2141033 and #2232813. This research is based upon work supported in part by the Office of the Director of National Intelligence (ODNI), Intelligence Advanced Research Projects Activity (IARPA), via Contract No. 2019-19020700008. This material is based upon work supported by the Defense Advanced Research Projects Agency, DARPA, under Air Force Contract No. FA8702-15-D-0001. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of DARPA. The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies, either expressed or implied, of ODNI, IARPA, or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for governmental purposes notwithstanding any copyright annotation therein.

References

- [AF19] Sohaib Ahmad and Benjamin Fuller. Unconstrained iris segmentation using convolutional neural networks. In ACCV 2018 Workshops, pages 450–466. Springer, 2019.
- [AG22] Megumi Ando and Marilyn George. On the cost of suppressing volume for encrypted multi-maps. Proceedings on Privacy Enhancing Technologies, 4, 2022.
- [APP⁺23] Ghous Amjad, Sarvar Patel, Giuseppe Persiano, Kevin Yeo, and Moti Yung. Dynamic volume-hiding encrypted multi-maps with applications to searchable encryption. PoPETS, 1:417–436, 2023.
- [BBOH96] Christopher M Brislawn, Jonathan N Bradley, Remigius J Onyshczak, and Tom Hopper. The FBI compression standard for digitized fingerprint images. In Proc. SPIE, volume 2847, pages 344–355, 1996.
- [BHJP14] Christoph Bösch, Pieter Hartel, Willem Jonker, and Andreas Peter. A survey of provably secure searchable encryption. ACM Computing Surveys (CSUR), 47(2):1–51, 2014.
- [BIPW17] Elette Boyle, Yuval Ishai, Rafael Pass, and Mary Wootters. Can we access a database both locally and privately? In TCC (2), pages 662–693. Springer, 2017.
- [BT21] Alexandra Boldyreva and Tianxin Tang. Privacy-preserving approximate k-nearest-neighbors search that hides access, query and volume patterns. PoPETS, 2021.
- [CCD⁺20] Hao Chen, Ilaria Chillotti, Yihe Dong, Oxana Poburinnaya, Ilya Razenshteyn, and M Sadegh Riazi. SANNS: Scaling up secure approximate k-Nearest neighbors search. In USENIX Security, pages 2111–2128, 2020.
- [CGKO06] Reza Curtmola, Juan Garay, Seny Kamara, and Rafail Ostrovsky. Searchable symmetric encryption: improved definitions and efficient constructions. In CCS, pages 79–88, 2006.
- [CGPR15] David Cash, Paul Grubbs, Jason Perry, and Thomas Ristenpart. Leakage-abuse attacks against searchable encryption. In CCS, pages 668–679, 2015.

- [Dau09] John Daugman. How iris recognition works. In The essential guide to image processing, pages 715–739. Elsevier, 2009.
- [Dau14] John Daugman. 600 million citizens of India are now enrolled with biometric id,”. SPIE newsroom, 7, 2014.
- [DHP21] Marc Damie, Florian Hahn, and Andreas Peter. A highly accurate query-recovery attack against searchable encryption using non-indexed documents. In USENIX Security, page 143–160, 2021.
- [FMC⁺15] Benjamin Fuller, Darby Mitchell, Robert Cunningham, Uri Blumenthal, Patrick Cable, Ariel Hamlin, Lauren Milechin, Mark Rabe, Nabil Schear, Richard Shay, et al. Security and privacy assurance research (SPAR) pilot final report. Technical report, MIT Lincoln Laboratory, 2015.
- [FMC⁺20] Francesca Falzon, Evangelia Anna Markatou, David Cash, Adam Rivkin, Jesse Stern, and Roberto Tamassia. Full database reconstruction in two dimensions. In Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, pages 443–460, 2020.
- [FMET22] Francesca Falzon, Evangelia Anna Markatou, Zachary Espiritu, and Roberto Tamassia. Range search over encrypted multi-attribute data. Proceedings of the VLDB Endowment, 16(4):587–600, 2022.
- [Fou] Electronic Frontier Foundation. Mandatory national ids and biometric databases.
- [FP22] Francesca Falzon and Kenneth G. Paterson. An efficient query recovery attack against a graph encryption scheme. In Vijayalakshmi Atluri, Roberto Di Pietro, Christian D. Jensen, and Weizhi Meng, editors, Computer Security – ESORICS 2022, pages 325–345, Cham, 2022. Springer International Publishing.
- [FVK⁺15] Ben A Fisch, Binh Vo, Fernando Krell, Abishek Kumarasubramanian, Vladimir Kolesnikov, Tal Malkin, and Steven M Bellovin. Malicious-client security in blind seer: a scalable private dbms. In IEEE S&P, pages 395–410. IEEE, 2015.
- [FVY⁺17] Benjamin Fuller, Mayank Varia, Arkady Yerukhimovich, Emily Shen, Ariel Hamlin, Vijay Gadeppally, Richard Shay, John Darby Mitchell, and Robert K Cunningham. SoK: Cryptographically protected database search. In IEEE S&P, pages 172–191. IEEE, 2017.
- [FWG⁺16] Zhangjie Fu, Xinle Wu, Chaowen Guan, Xingming Sun, and Kui Ren. Toward efficient multi-keyword fuzzy search over encrypted outsourced data with accuracy improvement. IEEE TIFS, 11(12):2706–2716, 2016.
- [GJW19] Zichen Gui, Oliver Johnson, and Bogdan Warinschi. Encrypted databases: New volume attacks against range queries. In CCS, page 361–378, 2019.
- [GKL⁺20] Paul Grubbs, Anurag Khandelwal, Marie-Sarah Lacharité, Lloyd Brown, Lucy Li, Rachit Agarwal, and Thomas Ristenpart. Pancake: Frequency smoothing for encrypted data stores. In 29th USENIX Security Symposium, pages 2451–2468, 2020.
- [GKM21] Marilyn George, Seny Kamara, and Tarik Moataz. Structured encryption and dynamic leakage suppression. In Eurocrypt, pages 370–396. Springer, 2021.
- [GLMP18] Paul Grubbs, Marie-Sarah Lacharité, Brice Minaud, and Kenneth G Paterson. Pump up the volume: Practical database reconstruction from volume leakage on range queries. In CCS, pages 315–331, 2018.
- [GLMP19] Paul Grubbs, Marie-Sarah Lacharité, Brice Minaud, and Kenneth G. Paterson. Learning to reconstruct: Statistical learning theory and encrypted database attacks. In In 2019 IEEE Symposium on Security and Privacy (SP), page 1067–1083, 2019.

- [GPP23] Zichen Gui, Kenneth G. Paterson, and Sikhar Patranabis. Rethinking searchable symmetric encryption. In IEEE S&P, 2023.
- [GPPW24] Zichen Gui, Kenneth G Paterson, Sikhar Patranabis, and Bogdan Warinschi. Swisse: System-wide security for searchable symmetric encryption. Proceedings on Privacy Enhancing Technologies, 2024.
- [GRS17] Paul Grubbs, Thomas Ristenpart, and Vitaly Shmatikov. Why your encrypted database is not secure. In Proceedings of the 16th workshop on hot topics in operating systems, pages 162–168, 2017.
- [GRS22] Venkatesan Guruswami, Atri Rudra, and Madhu Sudan. Essential Coding Theory. 2022.
- [GSB⁺17] Paul Grubbs, Kevin Sekniqi, Vincent Bindschaedler, Muhammad Naveed, and Thomas Ristenpart. Leakage-abuse attacks against order-revealing encryption. In Security and Privacy (SP), 2017 IEEE Symposium on, pages 655–672. IEEE, 2017.
- [HCD⁺25] Julie Ha, Chloe Cachet, Luke Demarest, Sohaib Ahmad, and Benjamin Fuller. Private eyes: Zero-leakage iris searchable encryption. In ACM Codaspy, 2025. <https://eprint.iacr.org/2023/736>.
- [HKR⁺24] Mahdiah Heidari pour, Ladan Kian, Maryam Rezapour, Mark Holcomb, Benjamin Fuller, Gagan Agrawal, and Hoda Maleki. Organizing records for retrieval in multi-dimensional range searchable encryption. Cryptology ePrint Archive, 2024.
- [HSWW18] Ariel Hamlin, Abhi Shelat, Mor Weiss, and Daniel Wichs. Multi-key searchable encryption, revisited. In PKC, pages 95–124. Springer, 2018.
- [IKK12] Mohammad Saiful Islam, Mehmet Kuzu, and Murat Kantarcioglu. Access pattern disclosure on searchable encryption: ramification, attack and mitigation. In NDSS, volume 20, page 12. Citeseer, 2012.
- [IM98] Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: towards removing the curse of dimensionality. In STOC, pages 604–613, 1998.
- [KE19] Evgenios M Kornaropoulos and Petros Efstathopoulos. The case of adversarial inputs for secure similarity approximation protocols. In 2019 IEEE European Symposium on Security and Privacy (EuroS&P), pages 247–262. IEEE, 2019.
- [KIK12] Mehmet Kuzu, Mohammad Saiful Islam, and Murat Kantarcioglu. Efficient similarity search over encrypted data. In IEEE Data Engineering, pages 1156–1167. IEEE, 2012.
- [KKM⁺22] Seny Kamara, Abdelkarim Kati, Tarik Moataz, Thomas Schneider, Amos Treiber, and Michael Yonli. Sok: Cryptanalysis of encrypted search with leaker - a framework for leakage attack evaluation on real-world data. In Euro S&P, 2022.
- [KKNO16] Georgios Kellaris, George Kollios, Kobbi Nissim, and Adam O’Neill. Generic attacks on secure outsourced databases. In CCS, pages 1329–1340, 2016.
- [KM19] Senny Kamara and Tarik Moataz. Computationally volume-hiding structured encryption. In EUROCRYPT 2019, volume 11477. Springer, 2019.
- [KMO18] Seny Kamara, Tarik Moataz, and Olya Ohrimenko. Structured encryption and leakage suppression. In Crypto, pages 339–370. Springer, 2018.
- [KP10] Ajay Kumar and Arun Passi. Comparison and combination of iris matchers for reliable personal authentication. Pattern recognition, 43(3):1016–1026, 2010.
- [KPT19] Evgenios M Kornaropoulos, Charalampos Papamanthou, and Roberto Tamassia. Data recovery on encrypted databases with k-nearest neighbor query leakage. In IEEE S&P, pages 1033–1050. IEEE, 2019.

- [KPT20] Evgenios M Kornaropoulos, Charalampos Papamanthou, and Roberto Tamassia. The state of the uniform: attacks on encrypted databases beyond the uniform query distribution. In IEEE S&P, pages 1223–1240, 2020.
- [LMWY20] Kasper Green Larsen, Tal Malkin, Omri Weinstein, and Kevin Yeo. Lower bounds for oblivious near-neighbor search. In SODA, pages 1116–1134. SIAM, 2020.
- [LPW⁺20] Qin Liu, Yu Peng, Jie Wu, Tian Wang, and Guojun Wang. Secure multi-keyword fuzzy searches with enhanced service quality in cloud computing. IEEE Transactions on Network and Service Management, 18(2):2046–2062, 2020.
- [LZWaT14] Chang Liu, Liehuang Zhu, Mingzhong Wang, and Yu an Tan. Search pattern leakage in searchable encryption: Attacks and new construction. In Information Sciences, volume 265, pages 176–188, 2014.
- [Mic97] Daniele Micciancio. Oblivious data structures: applications to cryptography. In STOC, page 456–464, New York, NY, USA, 1997. Association for Computing Machinery.
- [MT19] Evangelia Anna Markatou and Roberto Tamassia. Full database reconstruction with access and search pattern leakage. In International Conference on Information Security, pages 25–43. Springer, 2019.
- [OK21] Simon Oya and Florian Kerschbaum. Hiding the access pattern is not enough: Exploiting search pattern leakage in searchable encryption. In USENIX Security, 2021.
- [PPYY19] Sarvar Patel, Giuseppe Persiano, Kevin Yeo, and Moti Yung. Mitigating leakage in secure cloud-hosted data structures: Volume-hiding for multi-maps via hashing. In CCS, 2019.
- [RW23] Kui Ren and Cong Wang. Security impact of leakage profiles: Threats and countermeasures. In Searchable Encryption: From Concepts to Systems, pages 77–105. Springer, 2023.
- [SDF⁺25] Amey Shukla, Luke Demarest, Benjamin Fuller, Sohaib Ahmad, Caleb Manicke, Alexander Russell, and Sixia Chen. Fuzzy extractors are practical: Cryptographic strength key derivation from the iris. In CCS, 2025.
- [SDS⁺18] Emil Stefanov, Marten van Dijk, Elaine Shi, T-H Hubert Chan, Christopher Fletcher, Ling Ren, Xiangyao Yu, and Srinivas Devadas. Path oram: an extremely simple oblivious ram protocol. Journal of the ACM (JACM), 65(4):1–26, 2018.
- [SWP00] Dawn Xiaoding Song, David Wagner, and Adrian Perrig. Practical techniques for searches on encrypted data. In IEEE S&P, pages 44–55. IEEE, 2000.
- [WHL16] Boyang Wang, Yantian Hou, and Ming Li. Practical and secure nearest neighbor search on encrypted large-scale data. In INFOCOM, pages 1–9. IEEE, 2016.
- [WHL20] Boyang Wang, Yantian Hou, and Ming Li. Quickn: Practical and secure nearest neighbor search on encrypted large-scale data. IEEE Transactions on Cloud Computing, 10(3):2066–2078, 2020.
- [WLD⁺17] Guofeng Wang, Chuanyi Liu, Yingfei Dong, Hezhong Pan, Peiyi Han, and Binxing Fang. Query recovery attacks on searchable encryption based on partial knowledge. In International Conference on Security and Privacy in Communication Systems, pages 530–549. Springer, 2017.
- [WNL⁺14] Xiao Shaun Wang, Kartik Nayak, Chang Liu, TH Hubert Chan, Elaine Shi, Emil Stefanov, and Yan Huang. Oblivious data structures. In CCS, 2014.
- [WP21] Yun Wang and Dimitrios Papadopoulos. Multi-user collusion-resistant searchable encryption with optimal search time. In Asia CCS, pages 252–264, 2021.
- [WSL⁺22] Jianfeng Wang, Shi-Feng Sun, Tianci Li, Saiyu Qi, and Xiaofeng Chen. Practical volume-hiding encrypted multi-maps with optimal overhead and beyond. In CCS, page 2825–2839, New York, NY, USA, 2022. Association for Computing Machinery.

- [WYLH14] Bing Wang, Shucheng Yu, Wenjing Lou, and Y Thomas Hou. Privacy-preserving multi-keyword fuzzy search over encrypted data in the cloud. In IEEE INFOCOM, pages 2112–2120. IEEE, 2014.
- [ZKP16] Yupeng Zhang, Jonathan Katz, and Charalampos Papamanthou. All your queries are belong to us: The power of file-injection attacks on searchable encryption. In 25th USENIX Security Symposium, pages 707–720, 2016.
- [ZWX⁺23] Xianglong Zhang, Wei Wang, Peng Xu, Laurence T Yang, and Kaitai Liang. High recovery with fewer injections: practical binary volumetric injection attacks against dynamic searchable encryption. In USENIX Security, pages 5953–5970, 2023.

A Algorithmic Parameter Analysis for Random Data

This section compares the efficiency of the baseline scheme with **ProxCode**. During this discussion, we assume that all biometrics in the database are far yielding probability ϵ_f of their LSHs matching, and that all queries are closing yielding probability of ϵ_t of matching an LSH with the relevant stored record. These assumptions are useful for analysis but not true in practice, see discussion in Section 5. Our evaluation focuses on the number of required LSHs. We first compute accuracy parameters for **ProxCode** and then find corresponding parameters for the baseline scheme for the same accuracy.

A.1 Evaluation Methodology

We take the smallest values for α, n , and k that satisfy equations 4 and 5 simultaneously. Our parameter finding was done in Python 3.9.

Recall that for a (n, k) -Reed-Solomon Code to decode successfully (Definition 2) it suffices that $k_{correct} > 2k$ and $k_{error} \leq k$. Our evaluation uses an augmented LSH so we assume for some $\alpha \in \mathbb{Z}^+$ that $\epsilon_t = \epsilon_t'^\alpha$ and $\epsilon_f = \epsilon_f'^\alpha$.

We assume the bit selection LSH $LSH_i(x) = x_i$ which has the property that

$$\Pr[LSH(x) = LSH(y)] = \frac{\gamma - \mathcal{D}(x, y)}{\gamma} = 1 - \mathcal{D}(x, y)/\gamma.$$

We test with different parameters ϵ_t', ϵ_f' which represent the noise between different readings of the same biometric and readings of different biometrics respectively. Errors between readings of the same biometric and differences between readings of different biometrics both come from distributions. So for two different values $x_i, x_j \in \mathcal{DB}$, one will frequently observe $\mathcal{D}(x, y) < .5\gamma$. Even if the average FHD between readings of the same biometric is .1 one observes errors of at least .2. See Figure 3. This is why we test for values of $\epsilon_f' \in \{.5, .6, .7\}$. We consider $\epsilon_t' \in \{.7, .75, .80, .85, .9\}$. Our results exclude values where no solutions could be found with $\log_{10}(n) \leq 20$. We provide a full methodology next.

A.1.1 Detailed Methodology

For input constants c_1, c_2 we search for settings of α, n, k such that

$$(\epsilon_t')^\alpha > \frac{2k}{(1 - c_2)n}, \tag{7}$$

$$(\epsilon_f')^\alpha \leq \frac{k}{Mn(1 + c_1)}. \tag{8}$$

Increasing α exponentially decreases both the true accept rate and false accept rate. Thus, we first find the minimum α that produces a solution for n, k . Combining the Equations 7 and 8 one has that:

$$M(1 + c_1)(\epsilon_f')^\alpha \leq \frac{k}{n} < \frac{1}{2}(1 - c_2)(\epsilon_t')^\alpha.$$

We compute the minimum α such that

$$M \cdot (1 + c_1)(\epsilon'_f)^\alpha \leq \frac{1}{2}(1 - c_2)(\epsilon'_t)^\alpha$$

Using the computed α , we find the first n that satisfies the following inequality:

$$M \cdot (1 + c_1)(\epsilon'_f)^\alpha \cdot n \leq \left(\frac{1}{2}(1 - c_2)(\epsilon'_t)^\alpha + 1 \right) \cdot n$$

With α , and n we can easily find the set of possible as solutions to:

$$M \cdot (1 + c_1)(\epsilon'_f)^\alpha \cdot n \leq k < \frac{1}{2}(1 - c_2)(\epsilon'_t)^\alpha \cdot n. \quad (9)$$

As we show next the value of k , is strongly connected to the error probabilities in Lemma 1 and Lemma 2. Thus, we exclude solutions where $k < 20$ or k is not an integer.

Lastly, we check the probability that setup fails according to Lemma 3. We compute

$$\begin{aligned} \delta_{\text{Far}} &\leq \exp\left(\frac{-c_1^2}{2 + c_1} \cdot \epsilon'_f{}^\alpha \cdot n \cdot M\right) \approx \exp\left(\frac{-c_1^2 k}{(2 + c_1)(1 + c_1)}\right) \\ \delta_{\text{Close}} &\leq \exp\left(\frac{-c_2^2}{2} \cdot \epsilon'_t{}^\alpha \cdot n\right) + \delta_{\text{Far}} \approx \exp\left(\frac{-c_2^2 k}{1 - c_2}\right) + \delta_{\text{Far}} \end{aligned}$$

We estimated the minimum value of ℓ such that **Setup** has probability of at least .99 of completing within ℓ iterations using Equation 6.

Computing parameters for baseline scheme Recall for the baseline scheme described in Construction 1 one has $\delta_{\text{Far}} = (1 - \epsilon'_f{}^\alpha)^{nM}$ and $\delta_{\text{Close}} = (1 - \epsilon'_t{}^\alpha)^n$. We solve the following two equations to compute α and n in the Baseline scheme.

$$\text{FAR} = Mn\epsilon'_f{}^\alpha, \quad \delta_{\text{Close}} = (1 - \epsilon'_t{}^\alpha)^n$$

As mentioned above, we require the baseline scheme to have the same **FAR** as our δ_{Far} . This is a much weaker condition. For example, for a dataset of size $M = 10^6$ and $\delta_{\text{Far}} = 10^{-4}$ corresponds to a **FAR** $\approx 10^{-10}$.

A.2 Required Number of LSHs

Our parameter analysis focuses on three different database sizes when $M = 10^6, 10^4$ and $M = 10^3$ representing a country wide specialized database, a large organization, and a medium size organization.

Discussion The smaller the value of δ_{Far} the more **ProxCode** improves over the baseline scheme. Furthermore, the more noise is present, represented by a decrease in ϵ_t the more **ProxCode** improves over the baseline scheme.

For $\epsilon'_t = .9$ the difference in $\log n$ between **ProxCode** and in baseline scheme is negative (across all three accuracy regimes). As error increases, for example, ϵ'_t to .7, **ProxCode** presents major improvement. This improvement is largest in higher accuracy regime with $\delta_{\text{Far}} = 10^{-6}$ and smaller with $\delta_{\text{Far}} = 10^{-3}$. The gap between $\log n$ is similar across sizes of databases M though the absolute size has a strong dependence on M . The summary comparison is presented in Table 1.

For instance, looking at Table 5, setting $c_1 = 3$, and $c_2 = .4$ results in a high accuracy setting yielding:

- $\delta_{\text{Far}} = 10^{-4}$.
- δ_{Close} varies in size between the order of 10^{-4} and 10^{-3} .

The improvements are most pronounced when the gap between ϵ'_t and ϵ'_f is smallest. As an example, when $\epsilon'_f = .6$ and $\epsilon'_t = .75$ (represented in table 5), for $M = 10^4$ records in the Baseline scheme we need $n = 10^{12.1}$ (with LSHs of size $\alpha = 91$), **ProxCode** requires $n = 10^{8.4}$ (with $\alpha = 53$). There are some cases where there is a large difference between underlying LSH error rates ($\epsilon'_t = .9, \epsilon'_f = .5$) where **ProxCode** performs worse requiring approximately 60% more LSHs. These are “easy cases” when few LSHs are required. However, **ProxCode** often makes drastic improvements: when $\epsilon'_f = .6$ and $\epsilon'_t = .80$ one moves from almost a 100 million LSHs to a million. Improvements follow the same pattern for the setting of $M = 10^3$ and $M = 10^6$.

$\delta_{\text{Far}} = 10^{-3}$		$M = 10^6$				$M = 10^4$				$M = 10^3$			
		Baseline	ProxCode			Baseline	ProxCode			Baseline	ProxCode		
ϵ'_f	ϵ'_t	$\alpha \log n$	$\alpha \log n$	k	δ_{Close}	$\alpha \log n$	$\alpha \log n$	k	δ_{Close}	$\alpha \log n$	$\alpha \log n$	k	δ_{Close}
.7	.85	117	9.1	84	7.8×10^{-3}	93	7.4	60	6.1×10^{-3}	81	6.5	48	5.2×10^{-3}
.7	.9	91	5.0	65	4.8×10^{-3}	72	4.1	46	3.9×10^{-3}	63	3.7	37	3.5×10^{-3}
.6	.75	102	13.6	73	11.0×10^{-3}	81	10.9	52	8.3×10^{-3}	71	9.7	42	7.1×10^{-3}
.6	.8	79	8.5	57	7.5×10^{-4}	63	6.9	41	5.9×10^{-4}	55	6.2	33	5.1×10^{-4}
.6	.85	66	5.6	47	5.3×10^{-3}	53	4.7	34	4.3×10^{-4}	46	4.2	27	3.8×10^{-3}
.6	.9	56	3.4	40	3.7×10^{-3}	45	2.9	26	3.2×10^{-3}	39	2.6	23	2.9×10^{-3}
.5	.7	67	11.1	48	9.3×10^{-3}	49	8.4	34	7.1×10^{-3}	42	7.3	27	6.2×10^{-3}
.5	.75	56	7.8	40	6.9×10^{-3}	41	6.0	20	3.9×10^{-4}	35	5.2	23	4.8×10^{-4}
.5	.8	49	5.7	35	5.4×10^{-4}	38	4.7	25	4.4×10^{-4}	31	4.0	20	3.9×10^{-4}
.5	.85	43	3.9	31	4.2×10^{-4}	32	3.3	22	3.5×10^{-4}	28	3.1	18	3.3×10^{-4}
.5	.9	39	2.7	28	3.2×10^{-4}	29	2.4	20	2.9×10^{-4}	25	2.2	16	2.7×10^{-4}
$\delta_{\text{Far}} = 10^{-4}$		Baseline	ProxCode			Baseline	ProxCode			Baseline	ProxCode		
		$\alpha \log n$	$\alpha \log n$	k	δ_{Close}	$\alpha \log n$	$\alpha \log n$	k	δ_{Close}	$\alpha \log n$	$\alpha \log n$	k	δ_{Close}
.7	.85	129	9.9	85	7.8×10^{-3}	104	8.1	61	6.1×10^{-3}	92	7.2	49	5.2×10^{-3}
.7	.9	100	5.4	66	4.9×10^{-3}	81	4.5	47	3.9×10^{-3}	72	4.1	38	3.5×10^{-3}
.6	.75	112	14.8	74	11.1×10^{-3}	91	12.1	53	8.4×10^{-3}	81	10.9	43	7.2×10^{-3}
.6	.8	87	9.3	58	7.5×10^{-4}	71	7.7	42	6.0×10^{-4}	63	6.9	34	5.2×10^{-4}
.6	.85	72	5.9	48	5.3×10^{-4}	58	4.8	34	4.2×10^{-3}	52	4.5	28	3.9×10^{-3}
.6	.9	62	3.7	41	3.7×10^{-3}	51	3.3	30	3.3×10^{-4}	45	2.9	24	3.0×10^{-3}
.5	.7	74	12.2	49	9.4×10^{-3}	61	10.3	36	7.5×10^{-4}	54	9.2	29	6.4×10^{-4}
.5	.75	62	8.6	41	7.0×10^{-3}	51	7.3	30	5.7×10^{-4}	45	6.5	24	4.9×10^{-3}
.5	.8	53	5.9	35	5.2×10^{-3}	44	5.2	26	4.5×10^{-4}	39	4.7	21	4.0×10^{-4}
.5	.85	47	4.1	31	4.0×10^{-3}	39	3.7	23	3.6	34	3.2	18	3.1×10^{-3}
.5	.9	43	2.9	28	3.1×10^{-3}	36	2.8	21	3.0	34	2.6	17	2.8×10^{-4}
$\delta_{\text{Far}} = 10^{-6}$		Baseline	ProxCode			Baseline	ProxCode			Baseline	ProxCode		
		$\alpha \log n$	$\alpha \log n$	k	δ_{Close}	$\alpha \log n$	$\alpha \log n$	k	δ_{Close}	$\alpha \log n$	$\alpha \log n$	k	δ_{Close}
.7	.85	142	10.7	87	7.9×10^{-3}	119	9.2	63	6.2×10^{-3}	108	8.5	52	5.5×10^{-3}
.7	.9	110	5.8	67	4.9×10^{-3}	92	5.0	49	4.1×10^{-3}	83	4.6	40	3.7×10^{-3}
.6	.75	124	16.2	76	11.3×10^{-3}	104	13.8	55	8.7×10^{-3}	94	12.6	45	7.5×10^{-3}
.6	.8	96	10.0	59	7.6×10^{-3}	81	8.7	43	6.0×10^{-3}	73	7.9	35	5.2×10^{-3}
.6	.85	79	6.3	49	5.3×10^{-3}	67	5.6	36	4.5×10^{-4}	60	5.0	29	3.9×10^{-3}
.6	.9	68	3.8	42	3.8×10^{-3}	58	3.6	31	3.4×10^{-4}	52	3.3	25	3.0×10^{-3}
.5	.7	83	13.6	50	9.5×10^{-3}	69	11.5	37	7.6×10^{-3}	63	10.7	30	6.5×10^{-3}
.5	.75	69	9.5	42	7.1×10^{-3}	58	8.2	31	5.8×10^{-4}	52	7.4	25	5.0×10^{-3}
.5	.8	59	6.5	36	5.3×10^{-3}	50	5.7	26	4.3×10^{-3}	45	5.3	22	4.1×10^{-4}
.5	.85	53	4.7	30	3.5×10^{-3}	45	4.3	24	3.7	40	3.8	19	3.2×10^{-3}
.5	.9	48	3.2	29	3.2×10^{-3}	40	2.8	21	2.8	36	2.6	17	2.6×10^{-3}

Table 5: Parameters Comparison between our ProxCode and Baseline scheme where $M \cdot \epsilon'_f{}^\alpha \cdot n = \delta_{\text{Far}}$ and $(1 - \epsilon'_t{}^\alpha)^n = \delta_{\text{Close}}$. In ProxCode parameters are computed as in Section A. The numbers for α, n, k are the first found solutions. For the baseline scheme we measure FAR while in ProxCode we measure δ_{Far} this allows the baseline scheme to have more errors for the same accuracy. Accuracy $\delta_{\text{Far}} \approx 10^{-3}$ from $c_1 = 2$ and $c_2 = .4$. Accuracy $\delta_{\text{Far}} \approx 10^{-4}$ from $c_1 = 3, c_2 = .4$ and accuracy $\delta_{\text{Far}} \approx 10^{-6}$ from $c_1 = 5, c_2 = .4$. Logarithms are base 10.

Impact of reducing α Although our prime interest is to decrease n , we can see that we also have improvement in the value of α . This improvement is in all testing parameters. Current oblivious maps [BT21, HCD⁺25] build trees and obviously traverse them, the LSH values are used to decide which child to visit. Decreasing α allows one to use a tree with a larger branching factor. This in turn decreases the number of communication rounds. So decreasing α improves efficiency even if n remains the same.

Probability of setup completing Assuming ℓ to be the number of iterations for the Setup to succeed. For the setting of δ_{Far} , one has

$$\left(1 - \prod_{i=2}^M (1 - \delta_{\text{Far},i})\right)^\ell \leq \eta$$

where η is the probability of failure. Requires that

$$\ell \geq \frac{\log(\eta)}{\log\left(1 - \prod_{i=2}^M (1 - \delta_{\text{Far},i})\right)}.$$

For our choice of parameters δ_{Far} , and n , we always have

$$1 - \prod_{i=2}^M (1 - \delta_{\text{Far},i}) \approx 0$$

This behavior held true regardless of the size of the dataset, giving evidence that $\ell = 1$ suffices. We note that we performed this computation with floating point arithmetic and its known inaccuracies. For our implementation, Section 5, we do observe parameters where setup takes a multiple ≤ 10 iterations to succeed.